

CS543/ECE549 Assignment 3

Name: Maximo Gustavo Rojas Pelliccia

NetId: mgr9

Part 1: Homography estimation

A: Describe your solution, including any interesting parameters or implementation choices for feature extraction, putative matching, RANSAC, etc.

At first, I tried using the Harris implementation given to us in the starter code, but I found myself needing more good matches to calculate a correct Homography matrix. I had some matches, but mostly all of them were incorrect. My main problem was trying to decipher whether I was normalizing correctly or not the one-dimensional vectors got from the local neighborhood extraction. An interesting thing I encountered using this method, is that depending on the neighborhood size I chose, the amount of matches I got were shaped like a normal/gaussian distribution. A neighbourhood of 7x7 pixels was the optimal window that got me the most amount of matches between the two images. When increasing or decreasing the value, the matches dropped. It was also really tricky to get all the points I actually felt were good matches with a specific threshold as every time I tried to let out bad matches, I also was filtering good matches. I tried to fix my code to use the Harris detector, but to no avail, I finally decided to switch to SIFT descriptors.

Using SIFT descriptors, I started getting better results when matching key points under a threshold. To find matches, I first calculate the Euclidean distance between the points, and then, under a threshold, I loop around the points with their corresponding distance and filter those over the threshold.

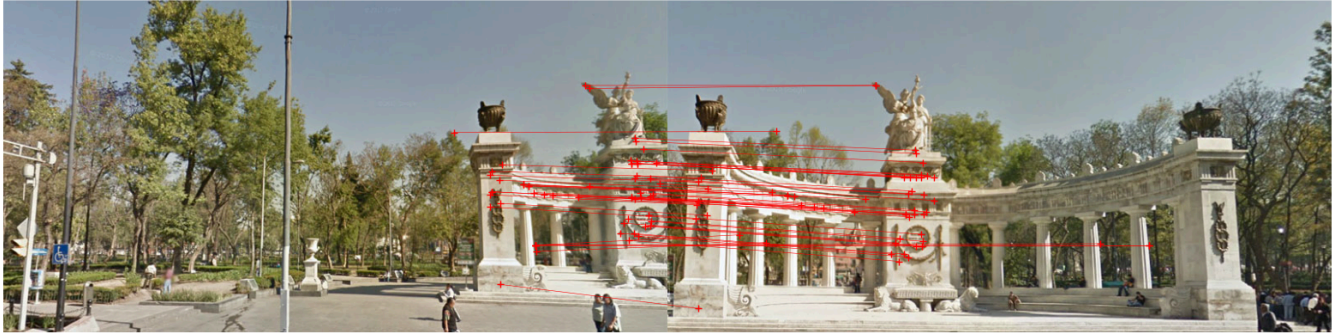
After having the matches, I calculate the homography Matrix which I use for the Ransac algorithm. Finally, with the best Homography matrix the for loop found with the Ransac algorithm, I start warping both images according to the transformation matrix. After I have both warped images, I stitch them together.

On one hand, the most difficult part of this assignment for me was actually warping both images correctly, which in my opinion, I did a poor job. I found this Stack Overflow thread [“python - How to show the whole image when using OpenCV warpPerspective - Stack Overflow”](#) which helped me visualize how to correctly warp and stitch the images.

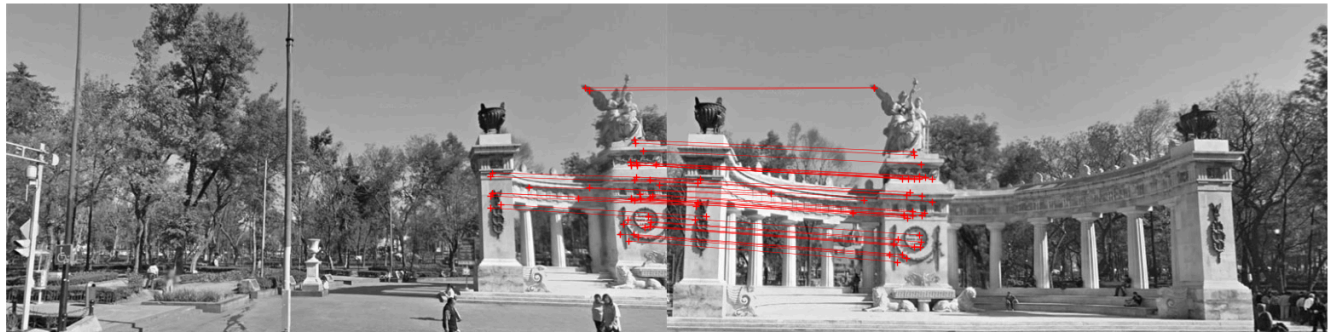
On the other hand, I still feel unsure about how my code calculates homography and my Ransac implementation seeing the results classmates posted on CampusWire, but I am happy with the results I got, especially on the three-image stitch. This assignment was interesting to work on to see how key points actually reflect on the images. While developing it, I was wondering if this technology is/was used for taking panoramic pictures.

B: For the image pair provided, report the number of homography inliers and the average residual for the inliers. Also, display the locations of inlier matches in both images.

These are the matches of key points found by the SIFT detector under a specific threshold for their Euclidean distance



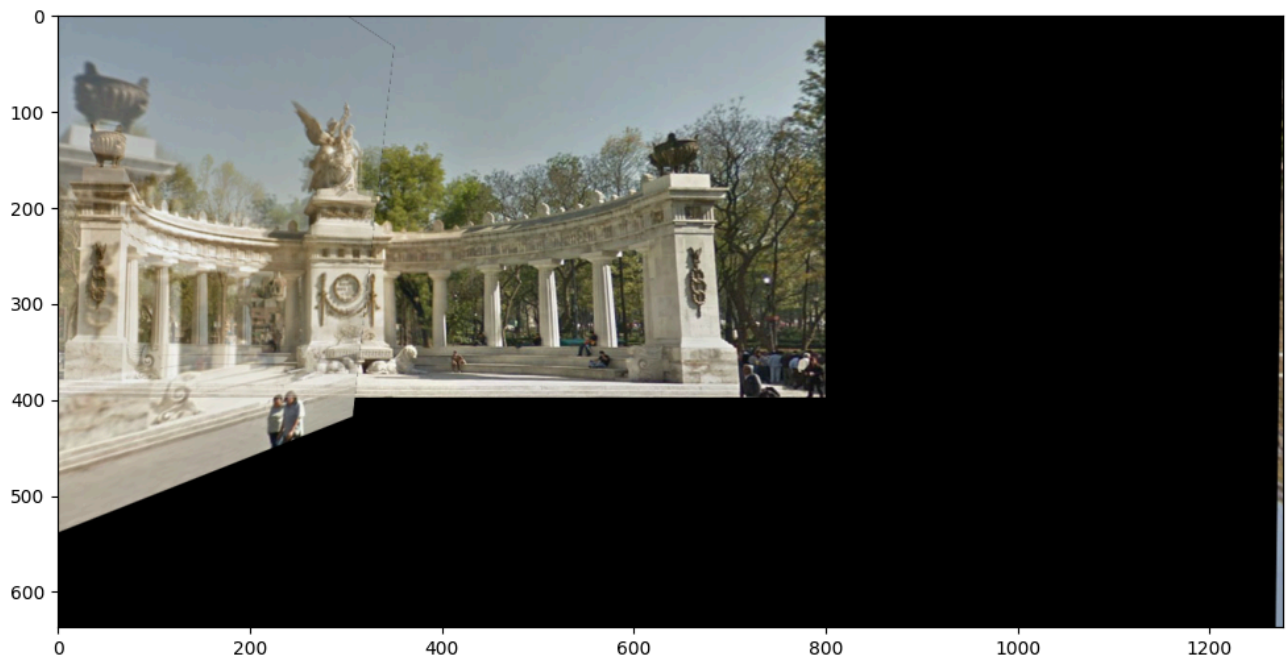
These are the Ransac inlier matches:



Average inlier residual: 4.5085455452165

Number of inliers: 43

C: Display the final result of your stitching.



Part 2: Stitching multiple images

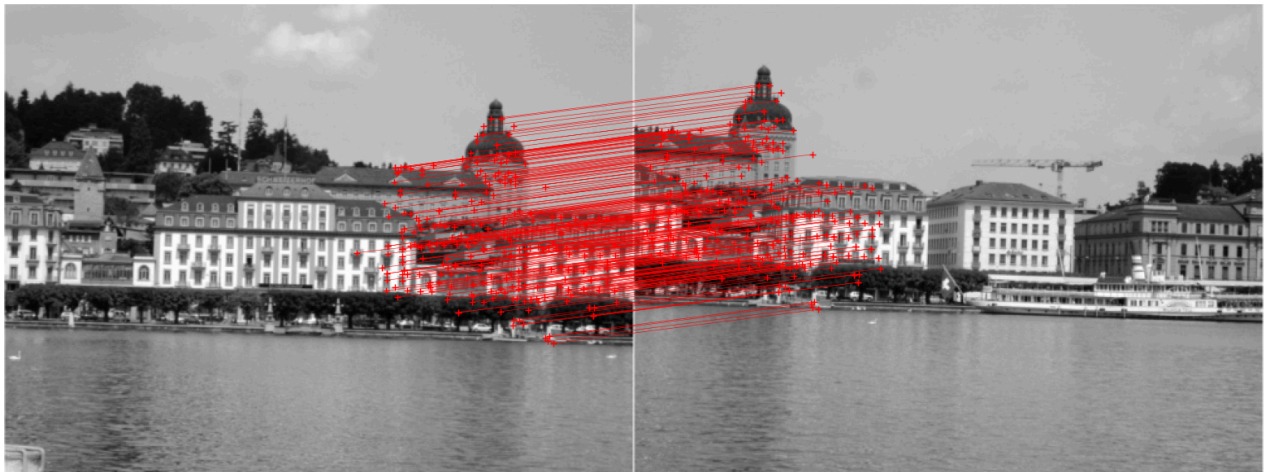
A: Describe your solution to the extent that it differs from Part 1.

The code used to compute the matches, homography, and Ransac are the same as of Part 1, the only thing differing is how it is calculated. I first calculated SIFT descriptors between images 1 and 2, 2 and 3, and filtered them. Then, I calculated the homography matrix and inliers for pairs 1 and 2, and 2 and 3. At first, I was trying to warp the images using these homography matrixes but was getting weird results. I later realized that I had to take into consideration image 1 as well to warp the stitched 1 and 2 images with image 3. For that, I used the dot product of both homography matrixes and used that as the homography transformation to end the stitching.

B: For the three images provided, report your order of stitching and how it was determined (it can be manual). For each pairwise registration, report the number of homography inliers and the average residual. There is *no need* to display inlier matches for this part.

The order of stitching for this part is: first images 1 and 2, using the homography matrix belonging to these two images. Then, stitched image 12 with image 3 using the dot product of homography matrixes of images 1 and 2 and 2 and 3.

For images 1 and 2:



Average inlier residual: 0.39537728372208136

Number of inliers: 269

For images 2 and 3:



Average inlier residual: 4.064183409755715

Number of inliers: 360

C: Display the final result of your stitching.

